

チェスゲーム システム設計書

Terminal UI / Pygame GUI / キャラクター演出の段階的拡張設計

東京理科大学 コンピュータ基礎 (TUS Computer Basic Class) | 開発言語: Python 3.12

作成日: 2026-10-01 | バージョン: 1.0 (正式改訂版)

目次

1 つくるもの (プロジェクト概要)	1
1.1 採用技術スタック	1
2 サクセスクリテリア (開発目標とマイルストーン)	2
3 設計メモの疑問に対する結論と設計方針	2
4 システムアーキテクチャ	3
4.1 ゲームループとデータフロー	3
5 詳細クラス設計	4
5.1 UI モジュール (chess_game.ui)	4
5.1.1 TerminalUI (Phase 1)	4
5.1.2 PygameGUI (Phase 2 & 3)	4
5.2 ゲームエンジン・駒モジュール (chess_game.core)	5
5.2.1 GameEngine	5
5.2.2 PieceEntity (コマメンバクラス)	5
5.3 プレイヤー・チェス AI モジュール (chess_game.players)	6
5.3.1 弱いアルゴリズム (RandomAIPlayer & GreedyAIPlayer)	6
5.3.2 強いアルゴリズム (MinimaxAIPlayer / α - β 枝刈り)	6
5.4 ランチャーモジュール (chess_game.launcher)	6
5.5 キャラクター・演出拡張モジュール (chess_game.character / Phase 3)	7
6 プロジェクトディレクトリ構成	7
7 画面・操作仕様	8
7.1 1. Terminal UI (Phase 1)	8
7.2 2. Pygame GUI (Phase 2 & 3)	8
8 実装スケジュールとマイルストーン	8
9 まとめ	9

1 つくるもの (プロジェクト概要)

本プロジェクトは、オブジェクト指向プログラミングの学習およびソフトウェア設計の段階的発展を実証するための**チェスゲームシステム**である。盤面ロジック・ユーザーインターフェース (UI)・対戦知能 (AI)・演出機能を明確に疎結合化し、テキスト端末 (CUI) からリッチな GUI、さらには 2 次元キャラクター演出を伴うソシャゲ風ゲームへとスムーズに拡張できるアーキテクチャを採用する。

1.1 採用技術スタック

区分	技術・ライブラリ	採用理由・役割
実行環境	Python 3.12 + uv	高速なパッケージ管理と型ヒント強化バージョンの利用
盤面・ルール管理	python-chess	厳密なチェス公式ルール (キャスリング、アンパッサン、50 手ルール、千日手など) の準拠と合法手生成

グラフィック・音声	pygame-ce	高速描画、クロスプラットフォーム動作、スプライト・SE再生の容易さ
テストフレームワーク	pytest	ルール判定、AI 着手、パーサーのユニットテスト自動化

2 サクセスクライテリア（開発目標とマイルストーン）

本開発では、仕様を 3 つのフェーズに分割し、各フェーズで単独完結した動作確認・テストが行える段階的デリバリーを採用する。

開発段階	目標名	達成条件・受入基準
第一段階	ミニマムサクセス (最低限の到達目標)	- Terminal UI (CUI/TUI) によるチェス対局の完走 - 標準入出力での手番入力 (UCI / SAN 形式) - 不正手の弾き・チェック/メイト判定 - ランダム AI との対局動作
第二段階	フルサクセス (高評価・標準目標)	- Pygame によるグラフィカル UI (GUI) - マウスクリック / ドラッグ&ドロップによる駒操作 - 合法手・直前着手の視覚ハイライト - ポーンプロモーション選択モーダル UI - Minimax 法+α-β枝刈りによる「手強い AI」の実装
第三段階	エクストラサクセス (発展・拡張目標)	2 次元素材を用いたキャラクター演出（駒スキン・立ち絵） - 王手時・駒捕獲時のカットイン演出とセリフ表示 - 対局報酬コインによるガチャシステム（駒スキンアンロック） - サウンド（BGM / SE / ボイス演出）の統合

3 設計メモの疑問に対する結論と設計方針

事前メモに記載されていたクラス設計の疑問点および設計選択について、ソフトウェア工学の観点から以下のように結論付けた。

Q (検討メモ): UI クラスは継承パターンにすべきか？ (Launcher から boot 起動できる IF があれば十分では？)

A (設計結論): 結論: 抽象基底クラス (BaseUI) による継承パターンを採用する。

Duck Typing により共通メソッド `boot(engine)` を持つだけでも動的ディスパッチは可能だが、以下の理由から継承が優れている：

- 契約の明文化:** ライフサイクル（起動、メインループ、描画、入力受付、終了）の抽象メソッドを定義し、実装漏れを実行前（静的解析・起動時）に検知できる。
- Template Method パターンの適用:** ターン処理の流れやイベント処理の基本スケルトンを基底クラスに共通化し、サブクラス (Terminal/GUI) は「描画」「入力取得」の具象化に専念できる。
- Launcher の単純化:** Launcher は具象クラスの違いを一切意識せず、ポリモーフィズムによって単一のコードで起動できる。

Q (検討メモ): チェス AI クラスは継承パターンか？**A (設計結論): 結論: まさに継承パターン (Strategy パターン) が最適解である。**

すべてのプレイヤーを抽象クラス Player から派生させ、AI だけでなく「人間プレイヤー」もその派生クラスとして設計する。

- Player 基底クラス: `get_move(board) -> chess.Move` を定義
- HumanPlayer: Terminal 入力や GUI マウスイベントから手を取得
- RandomAIPlayer (弱いアルゴリズム): 合法手からランダムに即答
- MinimaxAIPlayer (強いアルゴリズム): Minimax 探索 + α - β 枝刈り + 盤面評価関数

これにより、Engine は「相手が人間か AI か」を一切意識する必要がなくなり、「人 vs 人」「人 vs 弱 AI」「人 vs 強 AI」「AI vs AI (自己検証)」を同一コードで完全に切り替え可能となる。

Q (検討メモ): ゲームエンジンクラスにおける「コマメンバクラス」の役割は？**A (設計結論): 結論: python-chess の純粋駒情報と、UI・キャラクター演出用のメタデータを結ぶ「駒エンティティクラス (PieceEntity)」として設計する。**

python-chess 内部の駒は数値・記号の値オブジェクト (`chess.Piece`) であり、画像パスやアニメーション座標、キャラクター属性 (立ち絵・セリフ ID など) を持たない。そこで、ルール判定は python-chess に任せつつ、ゲームエンジンおよび描画層で「駒の個別 ID・スキン・状態」を管理するラッパークラスを設ける。これにより第 3 段階のキャラゲー拡張が極めて自然に統合できる。

4 システムアーキテクチャ

システム全体は関心の分離 (SoC) を徹底した 5 層のレイヤー構造で設計する。

レイヤー名	主要コンポーネント	責務
Presentation Layer (UI 層)	BaseUI, TerminalUI, PygameGUI	盤面の描画、ユーザー入力の検知、メッセージや演出の可視化
Application Layer (制御・起動層)	GameLauncher, GameController	コマンドライン引数解析、依存性注入 (DI)、ゲーム全体の起動制御
Domain Layer (ルール・状態層)	GameEngine, PieceEntity, BoardEvent	ルール遵守、合法手判定、手番遷移、イベント発行 (Observer)
Strategy Layer (知能・プレイヤー層)	Player, HumanPlayer, BaseAIPlayer, RandomAI, MinimaxAI	盤面状態に基づく次の 1 手 (<code>chess.Move</code>) の決定アルゴリズム
Expansion Layer (キャラ・演出拡張層)	CharacterManager, DialogueSystem, GachaSystem	2 次元素材、セリフ、カットイン演出、スキンアンロック管理

4.1 ゲームループとデータフロー

対局の 1 ターンにおけるメッセージのやりとりは以下の順序で進行する。

1. **着手要求:** GameEngine が現在の手番プレイヤー (Player) に `get_move(board)` を要求する。
2. **着手決定:**
 - 人間の場合: UI 層からの入力イベント (CUI テキストまたは GUI クリック) を待機して着手を返す。
 - AI の場合: アルゴリズム (ランダム / Minimax) により合法手から 1 手を計算して返す。
3. **検証と反映:** GameEngine が着手を検証 (合法手確認) し、盤面を更新する。

4. **イベント通知:** 駒取り (Capture)、王手 (Check)、終局 (Checkmate/Draw) などのイベントを UI 層・演出層へ通知する。
5. **再描画・演出:** UI 層が更新された盤面を描画し、該当イベントに応じたカットインやセリフ演出を表示する。

5 詳細クラス設計

5.1 UI モジュール (chess_game.ui)

UI クラス群は BaseUI を抽象基底クラス (ABC) として継承する。

```
from abc import ABC, abstractmethod
import chess
from chess_game.core.engine import GameEngine
from chess_game.core.events import GameEvent

class BaseUI(ABC):
    """すべてのUI (Terminal / GUI) の共通基底クラス"""

    def __init__(self, engine: GameEngine):
        self.engine = engine

    @abstractmethod
    def boot(self) -> None:
        """UIの初期化とメインループの開始"""
        pass

    @abstractmethod
    def render_board(self, board: chess.Board) -> None:
        """現在の盤面状態を描画する"""
        pass

    @abstractmethod
    def prompt_human_move(self, board: chess.Board) -> chess.Move:
        """人間プレイヤーからの入力を取得する"""
        pass

    @abstractmethod
    def on_game_event(self, event: GameEvent) -> None:
        """王手、駒取り、終局などのイベント通知を受け取る (Observer) """
        pass

    @abstractmethod
    def show_game_over(self, result_message: str) -> None:
        """対局終了画面・結果メッセージを表示する"""
        pass
```

5.1.1 TerminalUI (Phase 1)

- ・ **責務:** 標準入出力 (ANSI エスケープシーケンス利用可) での盤面描画とコマンド入力。
- ・ **特徴:**
 - ▶ Unicode チェス文字 (♔, ♚, ♜ 等) またはアルファベット (K, Q, P) による 8×8 盤面表示。
 - ▶ ユーザー入力は UCI 形式 (例: e2e4, e7e8q) および SAN 形式 (例: Nf3, O-O) をパース。
 - ▶ 不正手入力時には分かりやすいエラーメッセージを表示して再入力を促す。

5.1.2 PygameGUI (Phase 2 & 3)

- ・ **責務:** pygame-ce を用いたグラフィカルな盤面描画、マウス入力、アニメーション制御。
- ・ **特徴:**
 - ▶ マウスクリック選択およびドラッグ&ドロップによる直感的な操作。
 - ▶ 選択した駒の「移動可能な合法手マス」のハイライト表示 (半透明ドット)。
 - ▶ 直前の着手マス (From/To) の背景色ハイライト。
 - ▶ ポーンプロモーション時にクイーン・ルーク・ビショップ・ナイトを選択できるポップアップ UI。

- ・ 第3段階のカットイン・セリフ描画レイヤーをオーバーレイ可能。

5.2 ゲームエンジン・駒モジュール (chess_game.core)

5.2.1 GameEngine

チェス対戦の進行を完全にカプセル化する。

```
import chess
from typing import Callable, List
from chess_game.core.events import GameEvent, EventType
from chess_game.players.base import Player

class GameEngine:
    def __init__(self, white_player: Player, black_player: Player):
        self.board = chess.Board()
        self.white_player = white_player
        self.black_player = black_player
        self.listeners: List[Callable[[GameEvent], None]] = []

    def add_event_listener(self, listener: Callable[[GameEvent], None]) -> None:
        """イベントリスナー (UIや演出管理) を登録"""
        self.listeners.append(listener)

    def dispatch_event(self, event: GameEvent) -> None:
        for listener in self.listeners:
            listener(event)

    def current_player(self) -> Player:
        return self.white_player if self.board.turn == chess.WHITE else self.black_player

    def step(self) -> bool:
        """1手分の進行を行う。終局した場合はFalseを返す"""
        if self.board.is_game_over():
            return False

        player = self.current_player()
        move = player.get_move(self.board)

        # 着手前の状態判定 (駒取りの有無など)
        captured_piece = self.board.piece_at(move.to_square)
        self.board.push(move)

        # イベント発行
        if captured_piece:
            self.dispatch_event(GameEvent(EventType.CAPTURE, move=move, piece=captured_piece))
        if self.board.is_check():
            self.dispatch_event(GameEvent(EventType.CHECK, move=move))
        if self.board.is_game_over():
            self.dispatch_event(GameEvent(EventType.GAME_OVER, result=self.board.result()))

        return True
```

5.2.2 PieceEntity (コマメンバクラス)

python-chess の駒にビジュアル・キャラクター演出用のメタデータをバインドする。

```
from dataclasses import dataclass
from typing import Optional
import chess

@dataclass
class PieceEntity:
    """駒の表示・キャラクター属性を保持するエンティティ"""
    piece_type: chess.PieceType # PAWN, KNIGHT, etc.
```

```

color: chess.Color          # WHITE / BLACK
square: chess.Square        # 現在のマス座標
character_id: Optional[str] = None # 第3段階で紐付けるキャラID
skin_id: str = "default"    # 駒スキンID
is_captured: bool = False   # 取られた駒トレイにあるか

```

5.3 プレイヤー・チェス AI モジュール (chess_game.players)

AI アルゴリズムおよび人間プレイヤーを Strategy パターン（共通基底 Player）で統合する。

```

from abc import ABC, abstractmethod
import chess

class Player(ABC):
    def __init__(self, name: str, color: chess.Color):
        self.name = name
        self.color = color

    @abstractmethod
    def get_move(self, board: chess.Board) -> chess.Move:
        """与えられた盤面から次の1手を決定して返す"""
        pass

```

5.3.1 弱いアルゴリズム (RandomAIPlayer & GreedyAIPlayer)

- **RandomAIPlayer**: `list(board.legal_moves)` から `random.choice()` で一様ランダムに着手。探索コストゼロ、完全初心者・動作検証用。
- **GreedyAIPlayer** (初級): 1 手先の盤面において最も得点の高い駒取り手を選択する (Q=9, R=5, B=3, N=3, P=1)。安全な手であれば前進し、取れる駒がなければランダムに指す。

5.3.2 強いアルゴリズム (MinimaxAIPlayer / α - β 枝刈り)

チェス AI の王道アルゴリズムを実装し、確かな強さを提供する。

強いアルゴリズムの仕様要件

1. **探索手法**: Minimax 法 + α - β 枝刈り (Alpha-Beta Pruning)
2. **探索深さ**: デフォルト 3~4 手 (思考時間 1~3 秒程度で軽快に動作)
3. **Move Ordering (着手オーダリング)**: MVV-LVA (Most Valuable Victim - Least Valuable Attacker: 最も価値の低い駒で最も価値の高い駒を取る手) を優先して探索することで、枝刈り効率を劇的に向上させる。
4. **評価関数**: "Score" = sum ("Material") + sum ("Piece-Square Table (PST)") + "Mobility"

評価要素	重み・値	詳細説明
駒の素材価値 (Material)	P:100, N:320, B:330, R:500, Q:900, K:20000	盤上の残存戦力の単純合算。駒得を最優先。
配置テーブル (PST)	-50 ~ +50 点/マス	・ ナイト/ビショップ: 中央制圧マスで高得点 ・ キング: 序盤・中盤は端の安全地帯 (キャスリング先) で加点、終盤は中央進出で加点 ・ ポーン: 前進・中央支配で加点
モビリティ (Mobility)	1 手あたり +5 点	指せる合法手の選択肢の広さを評価 (窮屈な展開を避ける)

5.4 ランチャーモジュール (chess_game.launcher)

コマンドライン引数 (argparse) に応じて適切なクラスをインスタンス化 (依存性注入) し、アプリケーションを起動する。

```
# 起動コマンドの例:
# 1. ターミナルで人間 vs 弱AI
# python -m chess_game.launcher --ui terminal --white human --black random

# 2. GUIで人間 vs 強AI
# python -m chess_game.launcher --ui gui --white human --black minimax --depth 4

# 3. GUIでキャラ演出付き対戦
# python -m chess_game.launcher --ui gui --characters
```

5.5 キャラクター・演出拡張モジュール (chess_game.character / Phase 3)

ソシャゲ風のキャラクター性を持たせるためのデータ構造およびマネージャクラス。

- ・ Character: キャラクター ID、名前、レアリティ (★3~★5)、立ち絵画像、カットイン画像、ミニアイコン、セリフ辞書。
- ・ DialogueSystem: 「王手時」「駒撃破時」「自駒被撃破時」「勝利時」「敗北時」のシチュエーションに応じたセリフ (テキストおよび音声) の再生制御。
- ・ GachaSystem: 対局勝利時に獲得できるゲーム内コインを消費して、新しい駒スキン (キャラクター) を排出するアンロック機能。

6 プロジェクトディレクトリ構成

以下に、提案するソースコードの構成案を示す。

```
ChessGame/
├── pyproject.toml          # プロジェクト構成・依存関係定義
├── README.md              # 開発概要・起動手順
├── 設計書.typ             # 本システム設計書
├── assets/                # 静的リソースディレクトリ
│   ├── pieces/           # 標準チェス駒画像 (PNG / SVG)
│   ├── characters/       # 2次元キャラクター素材 (立ち絵・カットイン)
│   └── sounds/           # 効果音・BGM
├── src/
│   ├── chess_game/
│   │   ├── __init__.py
│   │   ├── main.py       # エントリーポイント
│   │   ├── launcher.py   # モード選択・クラス配線 (DI)・起動
│   │   ├── core/        # ドメイン・ルール層
│   │   │   ├── __init__.py
│   │   │   ├── engine.py # GameEngine (ルール進行、イベント発行)
│   │   │   ├── piece.py  # PieceEntity (駒メタデータ)
│   │   │   └── events.py  # GameEvent, EventType 定義
│   │   ├── players/     # プレイヤー・AI層
│   │   │   ├── __init__.py
│   │   │   ├── base.py   # Player 抽象基底クラス
│   │   │   ├── human.py  # HumanPlayer
│   │   │   ├── ai_random.py # RandomAIPlayer (弱いアルゴリズム)
│   │   │   └── ai_minimax.py # MinimaxAIPlayer (強いアルゴリズム)
│   │   └── ui/          # ユーザーインターフェース層
│   │       ├── __init__.py
│   │       ├── base.py   # BaseUI 抽象基底クラス
│   │       └── terminal/
│   │           ├── __init__.py
│   │           └── tui.py # TerminalUI
│   │       └── gui/
│   │           ├── __init__.py
│   │           ├── app.py # PygameGUI メイン
│   │           ├── board_view.py # 盤面・駒スプライト描画
│   │           └── dialog.py # プロモーション等のモータル
│   └── character/       # 第3段階拡張 (キャラ・ソシャゲ要素)
│       ├── __init__.py
│       └── model.py      # Character, Quote データモデル
```

```

├── cutin.py          # カットイン・演出制御
├── gacha.py          # ガチャ・スキン解放システム
└── tests/            # テストコード群
    ├── test_engine.py # ルール進行・イベントテスト
    ├── test_ai.py     # AIの合法手選択・探索テスト
    └── test_tui.py    # 入力パース・表示テスト

```

7 画面・操作仕様

7.1 1. Terminal UI (Phase 1)

- ・ **画面表示:** 毎ターン、現在の盤面を ASCII/Unicode で画面中央にクリア描画。上部に取られた駒一覧、手番情報 (White / Black)、直前の着手を表示。
- ・ **入力形式:**
 - ▶ UCI 表記: e2e4, g1f3, e7e8q (プロモーション時)
 - ▶ SAN 表記: e4, Nf3, 0-0 (python-chess の parse_san を利用して寛容に解釈)
 - ▶ コマンド: help (合法手一覧表示)、quit (対局中断)、undo (1 手戻す)
- ・ **エラー処理:** 非合法手や構文エラーが入力された場合、赤色テキストで「無効な着手です (例: e2e4)」と表示し、手番をスキップせず再入力を待つ。

7.2 2. Pygame GUI (Phase 2 & 3)

- ・ **解像度・レイアウト:** ウィンドウサイズ 960 × 640 px。左側に 600 × 600 px のチェス盤、右側に 360 px のステータス・キャラクター情報パネルを配置。
- ・ **マウス操作:**
 1. **クリック操作:** 自駒をクリックすると選択状態となり、移動可能マスに青いマーカーを表示。移動先をクリックすると着手完了。
 2. **ドラッグ&ドロップ:** 駒をドラッグして目的のマスで離すことでも着手可能。
- ・ **プロモーションモーダル:** ポーンが最奥列に達した際、ゲーム画面を薄暗くし、クイーン、ルーク、ビショップ、ナイトの 4 アイコンをポップアップ表示して選択させる。
- ・ **演出オーバーレイ (Phase 3):** 王手や駒取りが発生した際、盤面手前にキャラクターのカットイン画像がスライドインし、セリフ吹き出しと SE が再生される (数秒後にフェードアウトまたはクリックでスキップ)。

8 実装スケジュールとマイルストーン

工程	タスク名	主な作業内容
Step 1	基盤構築 & Terminal UI (ミニマムサクセス)	・ GameEngine, Player, BaseUI の抽象基底クラス定義 ・ python-chess と連携したターン進行・イベント発行 ・ TerminalUI の実装と手入力パーサー ・ RandomAIPlayer による単体対局の動作確認
Step 2	AI アルゴリズム強化 (強い AI の実装)	・ GreedyAIPlayer (初級) の実装 ・ MinimaxAIPlayer (Alpha-Beta 枝刈り + PST 盤面評価) の実装 ・ AI 同士の自己対戦ベンチマークによる強さ検証
Step 3	Pygame GUI 開発 (フルサクセス)	・ Pygame イベントループと 60FPS 描画機構 ・ 盤面・駒画像スプライト描画、マウス入力判定 ・ 合法手ハイライト、プロモーションモーダル実装 ・ ランチャーによる Terminal / GUI 切り替えの統合
Step 4	キャラクター・演出統合 (エクストラサクセス)	・ 2次元キャラ素材・スプライトシートの配置 ・ カットイン・セリフ描画 (CutinManager) ・ ゲーム内コインとガチャによる駒スキンアンロック機能 ・ BGM・効果音 (SE) の再生対応

9 まとめ

本設計により、事前メモで懸念されていた以下の点がすべて解決・整理された：

- ・ **UI の継承**: BaseUI を設けることで、Terminal と GUI を同一のインターフェースで安全に切り替え可能とし、ライフサイクルを統一した。
- ・ **AI の継承**: Strategy パターン (Player 派生) を導入することで、人間・弱 AI・強 AI を同一の視点で扱えるようになり、あらゆる対戦の組み合わせに拡張できた。
- ・ **駒クラスの配置**: 純粋ルール (python-chess) を汚さずに PieceEntity でリッチなキャラ情報を付与する二層構造とし、Phase 3 のキャラゲー化を破綻なく組み込める基盤を確立した。